

Assume the `sanitizedUrl` has been sanitized so that it is a true URL and not a JavaScript construct. Malicious users may also have CSS to “press jack” with the `userProvidedStyles`, such as styling the connection into a translucent box over the “Log in” tab. If <https://user-controlled-website.com/> is designed to look like the application’s login page, they could have just caught a user’s true login details.

You can see how enabling user-provided content for a `style` feature will increase the risk by granting that user complete power of how to style the whole website. As a result, Vue prevents the rendering of style tags inside models, such as:

- `<style>{{ userProvidedStyles }}</style>`

To keep your users secure from clickjacking, enable absolute CSS control only inside a sandboxed `iframe`. When providing user control with a style binding, utilize the object syntax and only enable users to have values for certain properties that are secure for them to control, such as this:

```
<a
  v-bind:href="sanitizedUrl"
  v-bind:style="{
    color: userProvidedColor,
    background: userProvidedBackground
  }"
  >
  click me
</a>
```

Copyright Attribution: vue.js

HTML template

Injecting JavaScript

It is strongly discouraged to use Vue to render a `script` feature since templates and render functions can never have side effects. This is not, though, the only way to use strings that would be evaluated as JavaScript at runtime.

Every HTML entity has attributes that embrace JavaScript strings as values, such as `onclick`, `onfocus`, and `onmouseenter`. Binding user-supplied JavaScript to any of these case attributes is dangerous and should be prevented.